Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Прогнозирование временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:
Студент 4 курса
Группы ИИ-26
Турич Д. А.
Проверил:
Никитин С. К.

Брест 2026

Цель работы

Исследовать нейросетевое прогнозирование временного ряда моделью p–N–1 при изменении размера окна и числа скрытых нейронов, сопоставить одношаговый и рекурсивный прогноз и оценить метрики на отложенных данных.

1. Вариант 6 и временной ряд

Заданы a=1,2; b=0,28; c=0,6; d=0,65; p=6; N=4. Формула ряда: $x(k)=1{,}2\cdot\cos(0{,}28k)+0{,}6\cdot\sin(0{,}65k)$, k=0,…,199. Первые 140 точек (k=0,…,139) — обучение, последние 60 (k=140,…,199) — тест.

Целевой x(k) предсказывается по окну [x(k−p),…,x(k−1)]. При p=6 создаются 134 обучающих примера (k=6,…,139) и 60 тестовых (k=140,…,199); тестовые окна могут включать известную историю обучающего периода без утечки тестовых целей.

Диапазон нормализации определяется только обучающим отрезком: $x_n=(x-x\min_train)/(x\max_train-x\min_train)$, xmin=-1.784001, xmax=1.782970. Обратное преобразование: $x=x\min_train+(x\max_train-x\min_train)\cdot x_n$. Использование min/max полного ряда вызвало бы утечку сведений о тестовом периоде.

2. Архитектура, градиенты и метрики

Сеть p–N–1: скрытая активация σ(z), линейный выход $y = hW_2+b_2$. $L=\frac{1}{2}(y-t)^2$; $\delta_2=y-t$, $\delta_1=(\delta_2 W_2^T)\odot h(1-h)$. Online backpropagation с перемешиванием примеров в каждой эпохе и малой случайной инициализацией U(−0,1;0,1). η=0,08 (внутри разрешённого диапазона 0,01–0,3). Критерий $E_s=\frac{1}{2}\text{mean}[(\hat{y}-t)^2]\le 10^{-4}$ или 20 000 эпох.

В оригинальной шкале: MAE=mean|ŷ−y|; RMSE=sqrt(mean[(ŷ−y)²]); $\text{sMAPE}=100\cdot\text{mean}[2|\hat{y}-y|/(|y|+|\hat{y}|)]$. Применена sMAPE, поскольку сумма гармоник проходит через ноль и обычная MAPE становится неустойчивой.

3. Базовый эксперимент p=6, N=4 (seed=6)

Эпохи	E_s	Train MAE	Train RMSE	Train sMAPE	Test MAE	Test RMSE	Test sMAPE
490	0.0000999	0.0411	0.0504	11.88%	0.0385	0.0476	15.09%

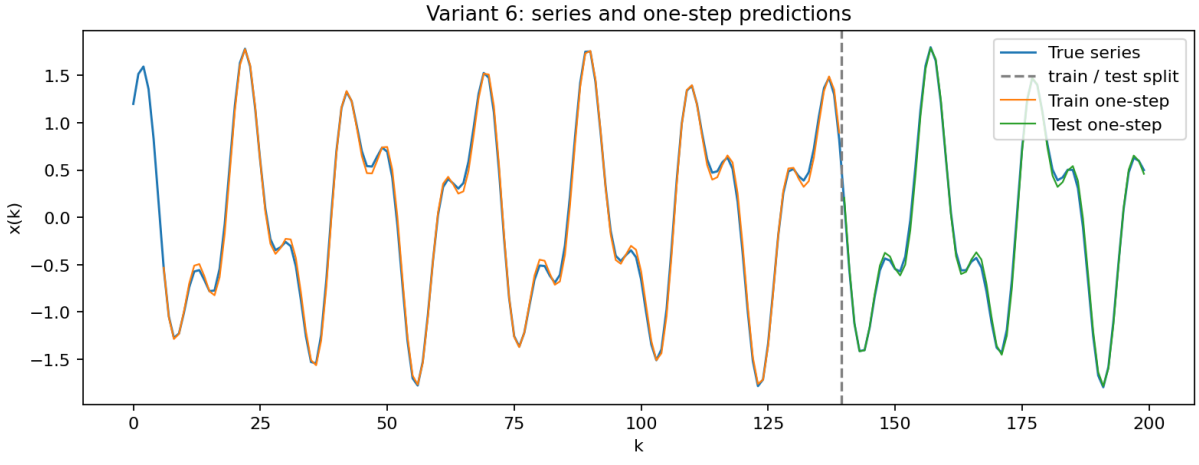


Рисунок 1 — Исходный ряд, граница обучения и одношаговый прогноз

4. Исследование длины окна p (N=4)

p	Эпохи: mean	Эпохи: std	Test MAE: mean	Test MAE: std	Не сошлись
2	20000.0	0.0	0.09774	0.00219	5
4	4890.2	85.4	0.03745	0.00032	0
6	507.2	18.9	0.03854	0.00040	0
12	36.2	2.2	0.03871	0.00165	0

5. Исследование скрытого слоя N (p=6)

N	Эпохи: mean	Эпохи: std	Test MAE: mean	Test MAE: std	Не сошлись
1	512.2	9.6	0.03861	0.00081	0
2	520.4	24.5	0.03821	0.00034	0
4	507.2	18.9	0.03854	0.00040	0
8	487.0	6.6	0.03814	0.00039	0

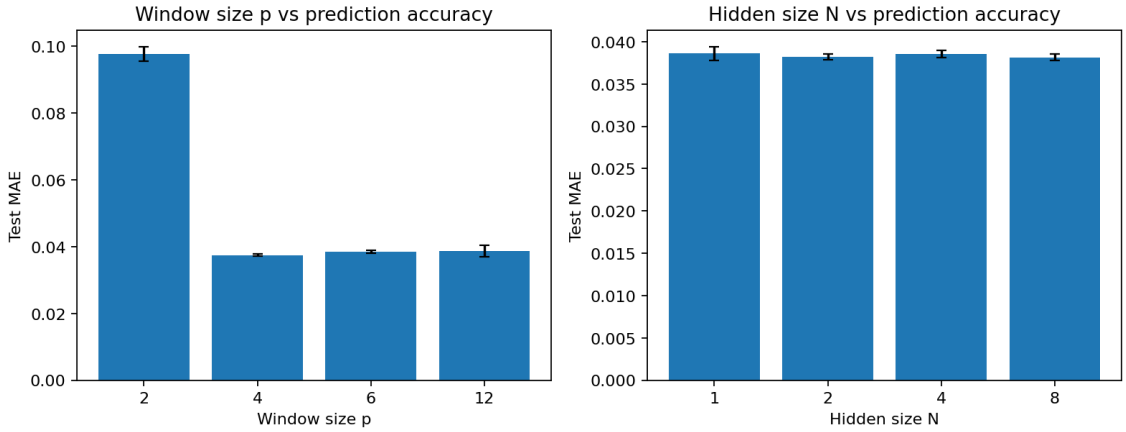


Рисунок 2 — Средняя ошибка прогноза и стандартное отклонение по пяти запускам

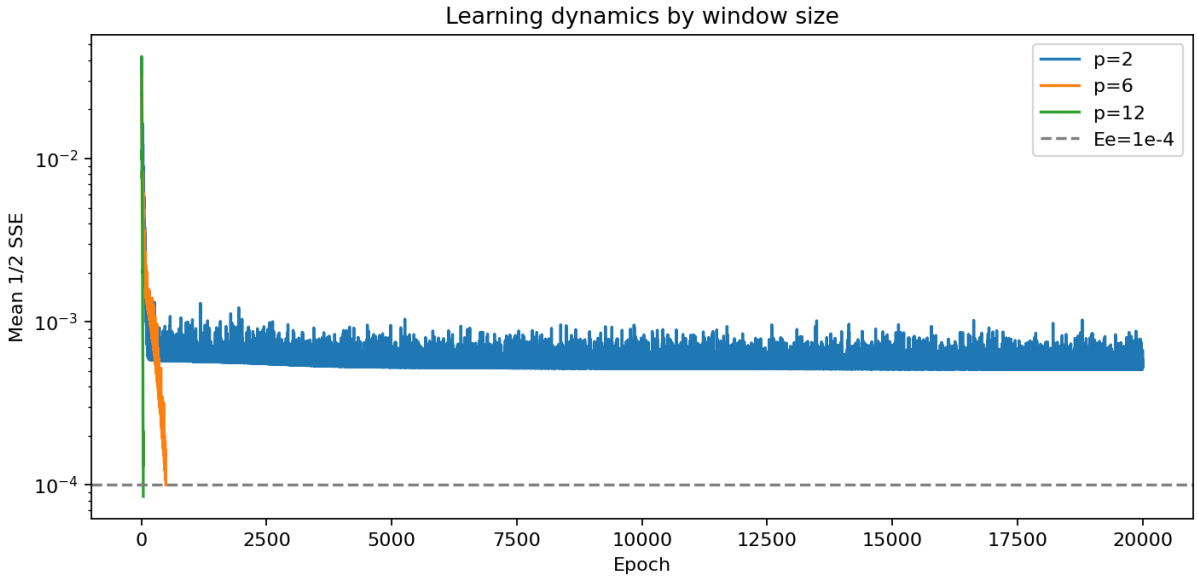


Рисунок 3 — Сходимость при окнах p=2, p=6 и p=12

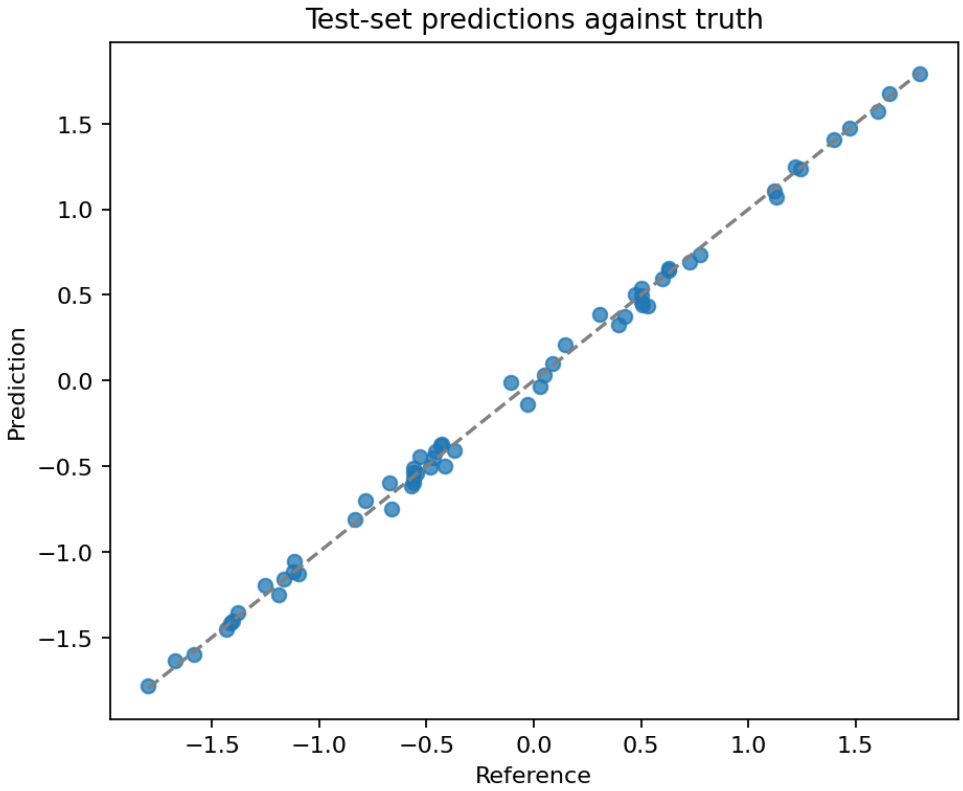


Рисунок 4 — Сравнение эталона и предсказания на тестовых точках

6. Рекурсивный прогноз 30 шагов

Лучший отдельный запуск по тестовой MAE: p=4, N=4, seed=7, MAE=0.036978. Для горизонта 30 запуск начинается с k=140 и подставляет каждое собственное предсказание в следующее окно. Порог 5% размаха всего ряда равен 0.179737. Первое превышение: шаг 3.

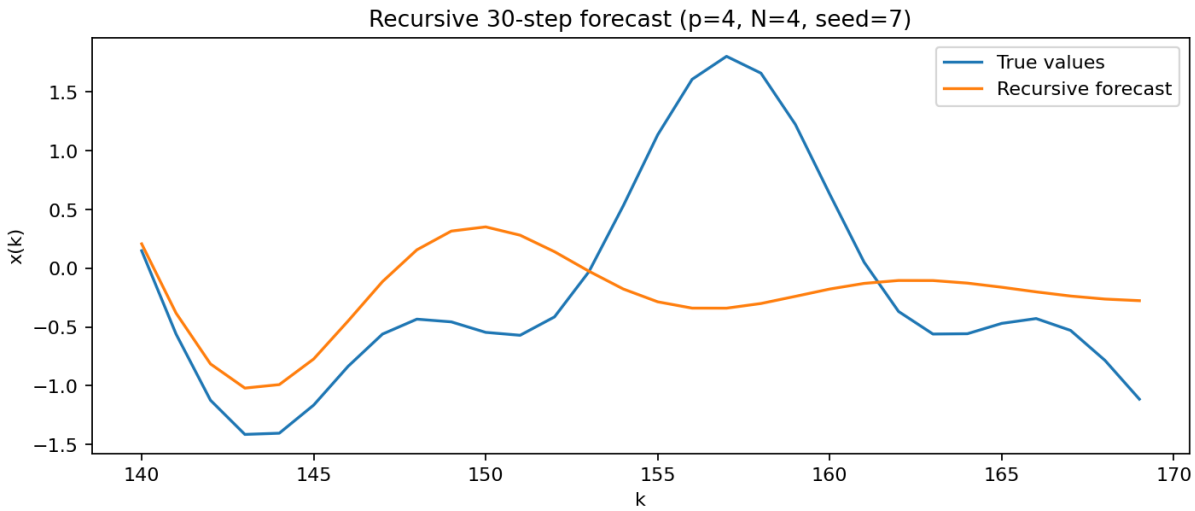


Рисунок 5 — Эталон и рекурсивный прогноз на горизонте 30

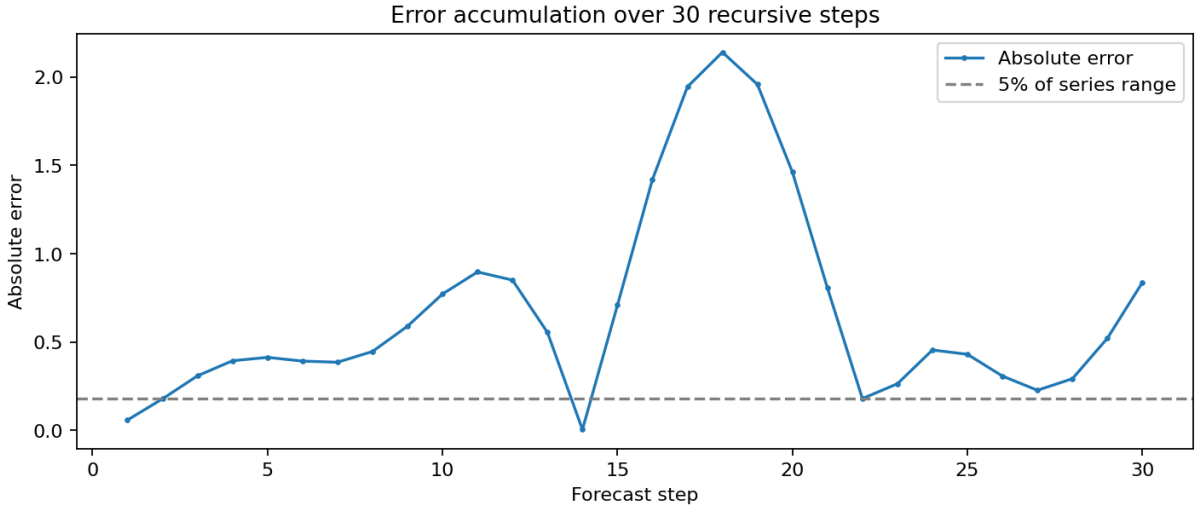


Рисунок 6 — Нарастание абсолютной ошибки по шагам

7. Режим функционирования

k / тип	ŷ норм.	ŷ исходное	Истинное	Абс. ошибка
20	0.8170	1.1301	1.1828	0.0527
40	0.6917	0.6834	0.7011	0.0178
100	0.3398	-0.5719	-0.6590	0.0871
140	0.5582	0.2071	0.1474	0.0596
160	0.6801	0.6418	0.6277	0.0141
199	0.6293	0.4606	0.5001	0.0395
ввод	0.8600	1.2836	—	—
с выбросом	1.0252	1.8729	—	—

Реализация также выводит все p нормализованных входов, а в интерактивном режиме принимает номер k и воспроизводит целевое значение, если оно известно. Пользовательские окна без ряда не имеют эталона, поэтому их абсолютная ошибка не вычисляется.

8. Дополнительное исследование устойчивости к шуму

Seed	MAE без шума	MAE с шумом	E_s с шумом	E_e достигнут
6	0.03848	0.09377	0.001263	нет
7	0.03816	0.10009	0.001514	нет
11	0.03806	0.11837	0.001299	нет
14	0.03890	0.10926	0.001016	нет
18	0.03907	0.10650	0.001489	нет

Контрольное испытание: на входы только обучающих окон добавлен гауссов шум $\sigma=0{,}03$ в нормализованной шкале, цели и тестовые окна оставлены без изменений. Для пяти инициализаций с шумом обучающая ошибка не достигла $E_e=10^{-4}$ за 20 000 эпох; средняя

ошибка тестового прогноза оказалась выше, чем без шума. Это показывает, что при данной постановке шум снижает точность; он не гарантирует улучшения обобщения.

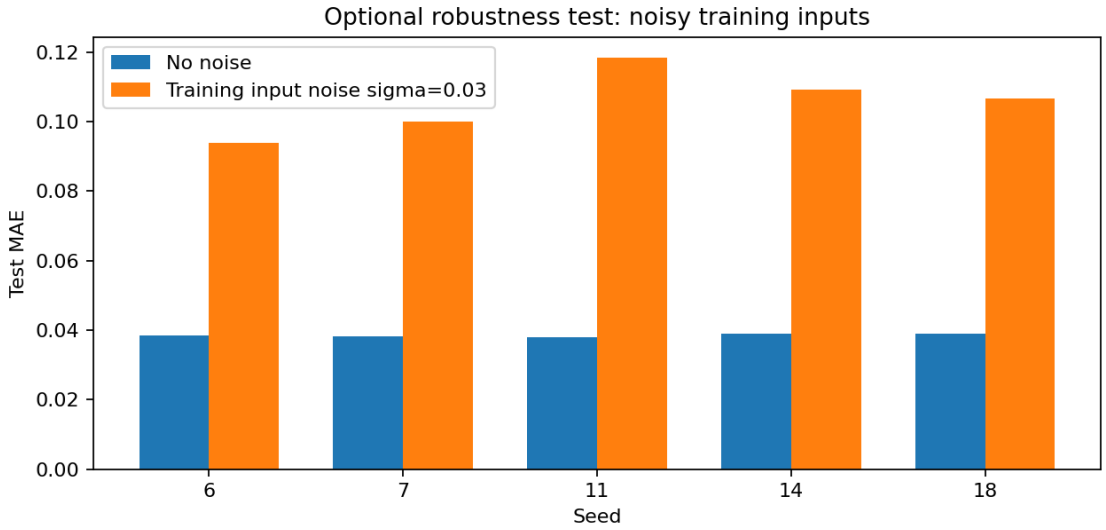


Рисунок 7 — Тестовая MAE в эксперименте с шумом

9. Приложение: подготовка рукописной части

По методичке требуются ФОТОГРАФИИ листов с рукописными вычислениями, где сверху собственноручно подписаны фамилия, инициалы, группа, вариант 6 и дата. Ниже приведены расчёты и схема для переписывания; машинописный текст НЕ является фотографией рукописи. После переписывания вставьте два реальных снимка вместо этого пояснения.

Пайплайн: генерация ряда → разделение train/test по времени → train Min-Max → нормализация → формирование скользящих окон → MLP p–N–1 → перемешивание после каждой эпохи → online backprop → остановка при $E_s \le 10^{-4}$ либо 20000 эпох → денормализация → MAE/RMSE/sMAPE → рекурсивный прогноз.

Нельзя использовать блоки Batch update, Sigmoid output, Normalize using whole dataset; для регрессии выход линейный, перемешивание применяется только к обучающим примерам, а min/max — исключительно к train.

Ручной расчёт сети 3–2–1, V=6, η=0.08: X=(0,2;0,5;0,8), t=0,6; W1=[[0,3;−0,2],[0,3;0,3],[−0,1;0,4]], b1=(0,1;−0,1); W2=(0,5;−1,2), b2=0,2.

Прямой проход: z_hidden=(0.230000; 0.330000); h=(0.557248; 0.581759); ŷ=-0.219487; L=½(ŷ−0,6)²=0.335780.

Обратный проход: δ_out=ŷ−0,6=-0.819487; δ_hidden=(-0.101093; 0.239273); ∂L/∂W_out=(-0.456658; -0.476744); ∂L/∂b_out=-0.819487.

Обновление: W_out ← W_out−η(h·δ_out); b_out←b_out−ηδ_out; W_hidden←W_hidden−η(X^T·δ_hidden); b_hidden←b_hidden−ηδ_hidden.

После шага: W_out=(0.536533;-1.161860); b_out=0.265559; b_hidden=(0.108087;-0.119142).

Вход	Новый вес на нейрон 1	Новый вес на нейрон 2
1	0.301617	-0.203828

Вход	Новый вес на нейрон 1	Новый вес на нейрон 2
2	0.304044	0.290429
3	-0.093530	0.384687

10. Выводы по вопросам методички

1) Увеличение окна с p=2 до p=4 резко улучшает качество. Окно p=2 не описывает достаточно полно комбинацию двух гармоник, а рост до 12 не гарантирует дальнейшего снижения MAE на тесте.

2) N=1 уже обеспечивает малую ошибку при p=6 благодаря достаточной информации в окне и нелинейному преобразованию; увеличение N не привело к устойчивому монотонному выигрышу, хотя потенциально повышает выразительность и риск переобучения.

3) Для регрессии линейный выход позволяет предсказывать значения вне [0;1]. Сигмоида ограничила бы диапазон и ухудшила бы обучение на ненормализованных целях.

4) Train Min-Max предотвращает утечку сведений из будущего. Тестовые значения могут выходить за [0;1]; это допустимый результат корректной процедуры и не повод подгонять нормализацию под тест.

5) Для p=2 в пяти запусках порог не был достигнут за 20 000 эпох; другие конфигурации достигли E_e. Эпохи и MAE менялись с seed, p и N при фиксированной η=0,08.

6) В рекурсивном прогнозе ошибка накапливается: модель потребляет собственные предсказания вместо истинных наблюдений. Одношаговая ошибка оценивает иную задачу; применимый горизонт определяется требованием к абсолютной ошибке.

7) Разница train/test описывает обобщающую способность: меньшая train-ошибка сама по себе не означает лучшее будущее предсказание. В дополнительном эксперименте шум σ=0,03 на тренировочных входах увеличил тестовую MAE, поэтому в данном примере он не улучшил обобщение; ранняя остановка, выбор N и увеличение выборки остаются возможными направлениями улучшения.

Листинг программы (полный исходный код)

В архиве код также сохранён отдельными редактируемыми файлами Python; ниже приведён полный листинг без скрытых библиотек машинного обучения.

Файл: ts_engine.py

```
001: """Time series MLP p-N-1 with online backpropagation (NumPy / Matplotlib)."""
002: import numpy as np
003:
004: def series():
005:     k=np.arange(200,dtype=float)
006:     return 1.2*np.cos(.28*k)+.6*np.sin(.65*k)
007:
008: def fit_scale(values):
009:     lo=float(values[:140].min());hi=float(values[:140].max());return lo,hi
010:
011: def normalize(v,lo,hi):return (np.asarray(v)-lo)/(hi-lo)
012: def denormalize(v,lo,hi):return lo+(hi-lo)*np.asarray(v)
013:
014: def windows(x,p):
015:     # For testing, prior observations from training are available, but targets never leak.
016:     xx=np.asarray([x[k-p:k] for k in range(p,len(x))],dtype=float)
017:     yy=x[p:].copy()
```

```
    inds=np.arange(p,len(x))
    return (xx[inds<140],yy[inds<140],inds[inds<140],
            xx[inds>=140],yy[inds>=140],inds[inds>=140])

def sigmoid(z):return 1/(1+np.exp(-np.clip(z,-60,60)))

class ForecastMLP:
    def __init__(self,p=6,n=4,seed=6,lr=.08):
        r=np.random.RandomState(seed)
        self.w1=r.uniform(-.1,.1,(p,n));self.b1=r.uniform(-.1,.1,n)
        self.w2=r.uniform(-.1,.1,n);self.b2=float(r.uniform(-.1,.1))
        self.p,self.n,self.lr=p,n,lr
    def predict(self,x):
        h=sigmoid(np.asarray(x)@self.w1+self.b1)
        return h@self.w2+self.b2
    def step(self,x,y):
        h=sigmoid(x@self.w1+self.b1)
        prediction=float(h@self.w2+self.b2)
        delta=prediction-y
        dh=delta*self.w2*h*(1-h)
        self.w2-=self.lr*delta*h
        self.b2-=self.lr*delta
        self.w1-=self.lr*np.outer(x,dh)
        self.b1-=self.lr*dh
    def fit(self,x,y,ee=1e-4,max_epochs=20000,seed=6):
        rng=np.random.RandomState(54321+seed)
        order=np.arange(len(y));history=[]
        for epoch in range(1,max_epochs+1):
            rng.shuffle(order)
            for j in order:self.step(x[j],y[j])
            pred=self.predict(x)
            es=float(.5*np.mean((pred-y)**2))
            history.append(es)
            if es<=ee:break
        return {'epochs':epoch,'Es':es,'converged':es<=ee,'history':np.array(history)}

def metrics(y,pred):
    err=np.asarray(pred)-np.asarray(y)
    mae=np.mean(abs(err));rmse=np.sqrt(np.mean(err**2))
    # Series crosses 0, so symmetric MAPE is used, with zero/zero treated as 0.
    den=np.abs(np.asarray(y))+np.abs(np.asarray(pred))
    smape=100*np.mean(np.where(den>1e-12,2*abs(err)/np.maximum(den,1e-12),0.))
    return dict(MAE=float(mae),RMSE=float(rmse),sMAPE=float(smape))

def recursive(model,observed_norm,start=140,steps=30):
    history=list(observed_norm[:start])
    preds=[]
    for _ in range(steps):
        v=float(model.predict(np.asarray(history[-model.p:])))
        preds.append(v);history.append(v)
    return np.asarray(preds)

def inference(model,source,lo,hi,k=None,custom=None):
    if custom is None:
        if k is None or not(model.p<=k<200):raise ValueError('Bad k')
        original=source[k-model.p:k]
        actual=source[k]
    else:
        original=np.asarray(custom,dtype=float)
        if len(original)!=model.p:raise ValueError('Provide exactly p values')
        actual=None
    xn=normalize(original,lo,hi)
    pred_norm=float(model.predict(xn))
    pred=float(denormalize(pred_norm,lo,hi))
    return
{'k':k,'inputs':original.tolist(),'inputs_norm':xn.tolist(),'pred_norm':pred_norm,
            'pred':pred,'true':None if actual is None else float(actual),
            'AE':None if actual is None else abs(pred-actual)}
```

Файл: main.py

```
"""Lab 4 (variant 6). Full 5-seed window/width studies; execution may take time."""
import argparse,json
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
from ts_engine import
series,fit_scale,normalize,denormalize,windows,ForecastMLP,metrics,recursive,inference
SEEDS=[6,7,11,14,18]
```

```
008:
009: def experiment(p,n,seed,epochs=20000):
010:     raw=series();lo,hi=fit_scale(raw);scaled=normalize(raw,lo,hi)
011:     xtr,ytr,itr,xte,yte,ite=windows(scaled,p)
012:     model=ForecastMLP(p,n,seed=seed,lr=.08)
013:     result=model.fit(xtr,ytr,ee=1e-4,max_epochs=epochs,seed=seed)
014:     train_metrics=metrics(raw[itr],denormalize(model.predict(xtr),lo,hi))
015:     test_pred=denormalize(model.predict(xte),lo,hi)
016:     test_metrics=metrics(raw[ite],test_pred)
017:     return model,result,train_metrics,test_metrics,ite,test_pred,raw,lo,hi
018:
019: def main(max_epochs=20000,interactive=False):
020:     configs=[(p,4) for p in (2,4,6,12)]+[(6,n) for n in (1,2,8)]
021:     outdir=Path(__file__).parent/'results';outdir.mkdir(exist_ok=True)
022:     summary={};best=None
023:     for p,n in configs:
024:         for seed in SEEDS:
025:             model,train,tr,te,k,y,raw,lo,hi=experiment(p,n,seed,max_epochs)
026:             key=f'{p}_{n}_{seed}'
027:             summary[key]={'p':p,'N':n,'seed':seed,'epochs':train['epochs'],
028:               'Es':train['Es'],'converged':train['converged'],
029:               'train':tr,'test':te}
030:             print(key,summary[key])
031:             if best is None or te['MAE']<best[0]:best=(te['MAE'],model,raw,lo,hi)
032:     with open(outdir/'latest_run.json','w',encoding='utf8') as
f:json.dump(summary,f,ensure_ascii=False,indent=2)
033:     _,model,raw,lo,hi=best
034:     rec=denormalize(recursive(model,normalize(raw,lo,hi),140,30),lo,hi)
035:     fig,ax=plt.subplots(figsize=(8,4))
036:     ax.plot(np.arange(140,170),raw[140:170],label='True')
037:     ax.plot(np.arange(140,170),rec,label='Recursive')
038:     ax.set(xlabel='k',ylabel='x(k)',title='30-step forecast');ax.legend()
039:     fig.tight_layout();fig.savefig(outdir/'recursive_runtime.png',dpi=160);plt.close(fig)
040:     if interactive:
041:         while True:
042:             v=input(f'Enter k or q (p={model.p}): ').strip()
043:             if v.lower()=='q':break
044:             try:print(inference(model,raw,lo,hi,k=int(v)))
045:             except ValueError as e:print(e)
046:
047: if __name__=='__main__':
048:     p=argparse.ArgumentParser();p.add_argument('--
quick',action='store_true');p.add_argument('--interactive',action='store_true')
049:     a=p.parse_args();main(500 if a.quick else 20000,a.interactive)
```

Файл: noise_study.py

```
001: """Optional experiment: input noise on the training set, never on test.
002: The Gaussian noise has sigma=0.03 in normalized input units.
003: Only NumPy is required; this may take a few minutes without acceleration.
004: """
005: import json
006: from pathlib import Path
007: import numpy as np
008: from ts_engine import (series,fit_scale,normalize,denormalize,windows,
009:                        ForecastMLP,metrics)
010:
011: def main():
012:     src=series();lo,hi=fit_scale(src)
013:     tr,targets,train_ids,test,_,test_ids=windows(normalize(src,lo,hi),6)
014:     records=[]
015:     for seed in (6,7,11,14,18):
016:         rng=np.random.RandomState(1000+seed)
017:         noisy=tr+rng.normal(loc=0.0,scale=0.03,size=tr.shape)
018:         network=ForecastMLP(6,4,seed=seed,lr=.08)
019:         result=network.fit(noisy,targets,ee=1e-4,max_epochs=20000,seed=seed)
020:         preds=denormalize(network.predict(test),lo,hi)
021:         records.append({'seed':seed,'epochs':result['epochs'],
022:               'Es':result['Es'],'converged':result['converged'],
023:               **metrics(src[test_ids],preds)})
024:         print(records[-1])
025:     with open(Path(__file__).parent/'results'/'noise_run.json','w',encoding='utf8') as f:
026:         json.dump(records,f,ensure_ascii=False,indent=2)
027:
028: if __name__=='__main__':main()
```